

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій

Кафедра САПР



Лабораторна робота №4

Частина 2

з дисципліни: “Застосування систем штучного інтелекту у технологічних рішеннях”

на тему:

“Генеративні моделі (GAN, VAE)”

Варіант №3

Виконав:

ст. групи ПП-44

Верещак Б. О.

Прийняв:

доц. Левкович М.В.

Мета роботи

Метою цієї частини є вивчення теоретичних основ генеративно-змагальних мереж (Generative Adversarial Networks, GAN) та їх архітектурного варіанту — глибоких згорткових GAN (Deep Convolutional GAN, DCGAN). Буде реалізовано модель DCGAN для генерації синтетичних зображень дорожніх знаків на основі датасету German Traffic Sign Recognition Benchmark (GTSRB). Основна увага приділяється аналізу процесу навчання, оцінці якості згенерованих зображень та дослідженню їх потенціалу для розширення (аугментації) навчальних даних.

Індивідуальне завдання

Виконати генерацію зображень за допомогою GAN на відкритому датасеті.

1. Завантажити підмножину даних (напр., 1–2 тис. зображень одного або кількох класів), зробити опис полів.
2. Підготувати дані: змінити розмір зображень (напр., 64x64), нормалізувати значення пікселів до діапазону $[-1, 1]$.
3. Навчити модель DCGAN з базовими гіперпараметрами.
4. Оцінити якість згенерованих зображень (візуально).
5. Побудувати візуалізації.
6. Зробити аналіз якості генерації, обговорити проблеми (напр., "колапс мод"), висновки щодо потенціалу для аугментації та пропозиції для покращення.
7. Висновки

Рекомендовані налаштування за замовчуванням:

- Архітектура (DCGAN): Генератор- Conv2DTranspose, BatchNormalization, LeakyReLU; Дискримінатор - Conv2D, LeakyReLU, Dropout.
- Оптимізатор: Adam з $\text{learning_rate}=0.0002$, $\text{beta_1}=0.5$.
- Функція втрат: BinaryCrossentropy.
- Навчання: $\text{batch_size}=128$, $\text{epochs}=50-100$.
- Метрики/Візуалізація: Графіки втрат генератора та дискримінатора; приклади згенерованих зображень на різних епохах.

№	Завдання	Метод	Тип даних	Результат	Датасет
3	Збільшення роздільної здатності знаків (SuperResolution)	SRGAN	Зображення знаків низької якості	Згенеровані зображення високої якості	German Traffic Sign Recognition Benchmark (GTSRB)

Хід роботи

1. Завантаження даних

Для цієї роботи я використав датасет German Traffic Sign Recognition Benchmark (GTSRB). Я завантажив його за допомогою Kaggle API (meowmeowmeowmeowmeow/gtsrb-german-traffic-sign). Цей набір даних містить понад 39,000 тренувальних зображень дорожніх знаків, розділених на 43 класи. У завданні пропонувалося взяти підмножину, тому я вирішив використати 3 перші класи на тренувальний набір. Датасет складається з .png зображень різного розміру та .csv файлів з анотаціями, але для цієї задачі я використовував лише самі зображення.

```
--- 1. Завантаження датасету 'meowmeowmeowmeowmeow/gtsrb-german-traffic-sign' ---
Using Colab cache for faster access to the 'gtsrb-german-traffic-sign' dataset.
Датасет успішно завантажено та скопійовано.
/kaggle/input/gtsrb-german-traffic-sign

import tensorflow as tf
from tensorflow.keras.layers import Input, Dense, Reshape, Flatten, Dropout,
BatchNormalization, LeakyReLU, Add, PReLU
from tensorflow.keras.layers import Conv2D, Conv2DTranspose, UpSampling2D
from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.applications import VGG19
from tensorflow.keras.losses import BinaryCrossentropy, MeanSquaredError

import matplotlib.pyplot as plt
import numpy as np
import os
import pathlib
import zipfile
import shutil
import kagglehub
KAGGLE_DATASET_NAME = "meowmeowmeowmeowmeow/gtsrb-german-traffic-sign"
UNZIP_DIR = pathlib.Path("")

def download_and_unzip_data():
    print(f"--- 1. Завантаження датасету '{KAGGLE_DATASET_NAME}' ---")
    try:
        # Використання системної команди kaggle
        path = kagglehub.dataset_download(KAGGLE_DATASET_NAME)
        path = pathlib.Path(path)
        print("Датасет успішно завантажено та скопійовано.")
        print(path)

    except Exception as e:
        print(f"Помилка під час завантаження або розпакування: {e}")
        return False
    return path

DATA_DIR = download_and_unzip_data()
```

2. Підготовка даних

Підготовка даних для SRGAN є ключовою і відрізняється від звичайної класифікації. Моделі потрібні пари зображень: низької якості (LR) та високої якості (HR). Спочатку я завантажив всі зображення і привів їх до єдиного розміру 64x64 пікселів – це будуть наші "ідеальні" HR-зображення. Потім я нормалізував їхні пікселі до діапазону $[-1, 1]$, як це рекомендовано для GAN, оскільки це допомагає стабілізувати навчання (використовуючи $\tanh$ як кінцеву активацію генератора). Далі, для кожного HR-зображення я створив відповідне LR-зображення: я зменшив його до 16x16, а потім збільшив назад до 64x64 за допомогою бікубічної інтерполяції. Таким чином, я отримав пари (розмите LR зображення 64x64, чітке HR зображення 64x64).

```
Збір тренувальних зображень з 3 класів
Знайдено 2220 зображень в класі '1'
Знайдено 2250 зображень в класі '2'
Знайдено 1410 зображень в класі '3'
Використовуюмо підмножину даних: 5880 зображень з 3 класів!
Тестовий набір містить 12630 зображень.
Створення конвеєра для 5880 зображень.
Створення конвеєра для 12630 зображень.
```

```
# --- 2. Підготовка даних ---
```

```
HR_SHAPE = (64, 64, 3)
LR_SHAPE = (16, 16, 3) # Зображення низької якості
UPSCALE_FACTOR = 4
BATCH_SIZE = 128
EPOCHS = 20
```

```
def load_image(image_path):
    img = tf.io.read_file(image_path)
    img = tf.image.decode_png(img, channels=3)
    img = tf.image.convert_image_dtype(img, tf.float32)
    return img

def create_lr_hr_pair(image):
    hr_image = tf.image.resize(image, [HR_SHAPE[0], HR_SHAPE[1]],
    method='bicubic')
    lr_image = tf.image.resize(hr_image, [LR_SHAPE[0], LR_SHAPE[1]],
    method='bicubic')
    hr_image = (hr_image * 2.0) - 1.0
    lr_input = tf.image.resize(lr_image, [HR_SHAPE[0], HR_SHAPE[1]],
    method='bicubic')
    lr_input = (lr_input * 2.0) - 1.0
    return lr_input, hr_image

def create_dataset_pipeline(image_paths_list):
    print(f"Створення конвеєра для {len(image_paths_list)} зображень.")
    AUTOTUNE = tf.data.AUTOTUNE

    ds = tf.data.Dataset.from_tensor_slices(image_paths_list)
    ds = ds.map(load_image, num_parallel_calls=AUTOTUNE)
    ds = ds.map(create_lr_hr_pair, num_parallel_calls=AUTOTUNE)
```

```

ds =
ds.batch(BATCH_SIZE).shuffle(buffer_size=256).prefetch(buffer_size=AUTOTUNE)
return ds

```

3. Навчання моделі SRGAN

Я реалізував модель SRGAN, яка складається з двох окремих мереж: Генератора та Дискримінатора.

```

# --- 3. Побудова моделі SRGAN ---
bce_loss = BinaryCrossentropy()
mse_loss = MeanSquaredError()

```

Генератор: Його завдання – брати розмите LR-зображення (64x64) і перетворювати його на чітке SR-зображення (64x64), яке виглядає як справжнє. Архітектура генератора – це глибока згорткова мережа з залишковими блоками (ResNet-подібна). Вона містить декілька залишкових блоків, кожен з яких складається з Conv2D, BatchNormalization та PReLU (Parametric ReLU, схоже на LeakyReLU). Наприкінці мережі я використав два блоки Conv2DTranspose (або UpSampling2D) для підвищення роздільної здатності (хоча в моєму випадку вхід і вихід одного розміру, ця архітектура легко адаптується і для реального апскейлінгу).

```

--- 3.a. Побудова Генератора ---
--- Модель Генератора успішно створена ---
Model: "functional"

```

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 64, 64, 3)	0	-
conv2d (Conv2D)	(None, 64, 64, 64)	15,616	input_layer[0][0]
p_re_lu (PReLU)	(None, 64, 64, 64)	64	conv2d[0][0]
conv2d_1 (Conv2D)	(None, 64, 64, 64)	36,928	p_re_lu[0][0]
batch_normalization (BatchNormalizatio...	(None, 64, 64, 64)	256	conv2d_1[0][0]
p_re_lu_1 (PReLU)	(None, 64, 64, 64)	64	batch_normalizat...
conv2d_2 (Conv2D)	(None, 64, 64, 64)	36,928	p_re_lu_1[0][0]
batch_normalization... (BatchNormalizatio...	(None, 64, 64, 64)	256	conv2d_2[0][0]
add (Add)	(None, 64, 64, 64)	0	p_re_lu[0][0], batch_normalizat...
conv2d_3 (Conv2D)	(None, 64, 64, 64)	36,928	add[0][0]
batch_normalization...	(None, 64, 64, 64)	256	conv2d_3[0][0]

(BatchNormalizatio...	64)		
p_re_lu_2 (PReLU)	(None, 64, 64, 64)	64	batch_normalizat...
conv2d_4 (Conv2D)	(None, 64, 64, 64)	36,928	p_re_lu_2[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 64, 64, 64)	256	conv2d_4[0][0]
add_1 (Add)	(None, 64, 64, 64)	0	add[0][0], batch_normalizat...
conv2d_5 (Conv2D)	(None, 64, 64, 64)	36,928	add_1[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 64, 64, 64)	256	conv2d_5[0][0]
p_re_lu_3 (PReLU)	(None, 64, 64, 64)	64	batch_normalizat...
conv2d_6 (Conv2D)	(None, 64, 64, 64)	36,928	p_re_lu_3[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 64, 64, 64)	256	conv2d_6[0][0]
add_2 (Add)	(None, 64, 64, 64)	0	add_1[0][0], batch_normalizat...
conv2d_7 (Conv2D)	(None, 64, 64, 256)	147,712	add_5[0][0]
p_re_lu_6 (PReLU)	(None, 64, 64, 256)	256	conv2d_12[0][0]
conv2d_8 (Conv2D)	(None, 64, 64, 3)	62,211	p_re_lu_6[0][0]

Total params: 635,203 (2.42 MB)

Trainable params: 633,795 (2.42 MB)

Non-trainable params: 1,408 (5.50 KB)

--- 3.a. Генератор ---

```
def residual_block(x):
    res = Conv2D(filters=64, kernel_size=3, strides=1, padding='same')(x)
    res = BatchNormalization(momentum=0.8)(res)
    res = PReLU(shared_axes=[1, 2])(res)
    res = Conv2D(filters=64, kernel_size=3, strides=1, padding='same')(res)
    res = BatchNormalization(momentum=0.8)(res)
    x = Add()([x, res])
    return x

def build_generator():
    print("--- 3.a. Побудова Генератора ---")
    input_lr = Input(shape=HR_SHAPE)
    c1 = Conv2D(64, kernel_size=9, strides=1, padding='same')(input_lr)
    c1 = PReLU(shared_axes=[1, 2])(c1)
    res = c1
    for _ in range(5):
        res = residual_block(res)
```

```

c2 = Conv2D(64, kernel_size=3, strides=1, padding='same')(res)
c2 = BatchNormalization(momentum=0.8)(c2)
c2 = Add()([c2, c1])
c3 = Conv2D(256, kernel_size=3, strides=1, padding='same')(c2)
c3 = PReLU(shared_axes=[1, 2])(c3)
gen_output = Conv2D(3, kernel_size=9, strides=1, padding='same',
activation='tanh')(c3)
model = Model(input_lr, gen_output)
print("---- Модель Генератора успішно створена ----")
model.summary()
return model

```

Дискримінатор: Це класична CNN, схожа на ту, що використовувалася у попередній лабораторній. Його завдання – подивитися на зображення (або справжнє HR, або згенероване SR) і сказати, чи воно "справжнє" (1) чи "фейкове" (0). Він складається з шарів Conv2D з активацією LeakyReLU та Dropout, як і було рекомендовано.

```

--- 3.b. Побудова Дискримінатора ---
--- Модель Дискримінатора успішно створена ---
Model: "functional_1"

```

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 64, 64, 3)	0
conv2d_14 (Conv2D)	(None, 64, 64, 64)	1,792
leaky_re_lu (LeakyReLU)	(None, 64, 64, 64)	0
conv2d_15 (Conv2D)	(None, 32, 32, 64)	36,928
leaky_re_lu_1 (LeakyReLU)	(None, 32, 32, 64)	0
batch_normalization_11 (BatchNormalization)	(None, 32, 32, 64)	256
conv2d_16 (Conv2D)	(None, 32, 32, 128)	73,856
leaky_re_lu_2 (LeakyReLU)	(None, 32, 32, 128)	0
batch_normalization_12 (BatchNormalization)	(None, 32, 32, 128)	512
conv2d_17 (Conv2D)	(None, 16, 16, 128)	147,584
leaky_re_lu_3 (LeakyReLU)	(None, 16, 16, 128)	0
batch_normalization_13 (BatchNormalization)	(None, 16, 16, 128)	512
conv2d_18 (Conv2D)	(None, 16, 16, 256)	295,168
leaky_re_lu_4 (LeakyReLU)	(None, 16, 16, 256)	0
batch_normalization_14 (BatchNormalization)	(None, 16, 16, 256)	1,024
conv2d_19 (Conv2D)	(None, 8, 8, 256)	590,080
leaky_re_lu_5 (LeakyReLU)	(None, 8, 8, 256)	0
batch_normalization_15	(None, 8, 8, 256)	1,024

(BatchNormalization)		
conv2d_20 (Conv2D)	(None, 8, 8, 512)	1,180,160
leaky_re_lu_6 (LeakyReLU)	(None, 8, 8, 512)	0
batch_normalization_16 (BatchNormalization)	(None, 8, 8, 512)	2,048
conv2d_21 (Conv2D)	(None, 4, 4, 512)	2,359,808
leaky_re_lu_7 (LeakyReLU)	(None, 4, 4, 512)	0
batch_normalization_17 (BatchNormalization)	(None, 4, 4, 512)	2,048
flatten (Flatten)	(None, 8192)	0
dense (Dense)	(None, 1024)	8,389,632
leaky_re_lu_8 (LeakyReLU)	(None, 1024)	0
dropout (Dropout)	(None, 1024)	0
dense_1 (Dense)	(None, 1)	1,025

Total params: 13,083,457 (49.91 MB)

Trainable params: 13,079,745 (49.90 MB)

Non-trainable params: 3,712 (14.50 KB)

--- 3.b. Дискримінатор ---

```
def build_discriminator():
    print("--- 3.b. Побудова Дискримінатора ---")
    input_img = Input(shape=HR_SHAPE)

    def d_block(layer_input, filters, strides=1, bn=True):
        d = Conv2D(filters, kernel_size=3, strides=strides,
padding='same')(layer_input)
        d = LeakyReLU(alpha=0.2)(d)
        if bn:
            d = BatchNormalization(momentum=0.8)(d)
        return d

    d1 = d_block(input_img, 64, bn=False)
    d2 = d_block(d1, 64, strides=2)
    d3 = d_block(d2, 128)
    d4 = d_block(d3, 128, strides=2)
    d5 = d_block(d4, 256)
    d6 = d_block(d5, 256, strides=2)
    d7 = d_block(d6, 512)
    d8 = d_block(d7, 512, strides=2)
    d9 = Flatten()(d8)
    d10 = Dense(1024)(d9)
    d10 = LeakyReLU(alpha=0.2)(d10)
    d10 = Dropout(0.4)(d10)
    validity = Dense(1, activation='sigmoid')(d10)
    model = Model(input_img, validity)
    print("--- Модель Дискримінатора успішно створена ---")
    model.summary()
    return model
```



```
# --- 3.c. VGG (Perceptual Loss) ---
def build_vgg():
    print("--- 3.c. Побудова VGG19 (для Perceptual Loss) ---")
    vgg = VGG19(weights="imagenet", include_top=False, input_shape=HR_SHAPE)
    vgg.trainable = False
    model = Model(inputs=vgg.input,
outputs=vgg.get_layer("block3_conv4").output)
    print("--- Модель VGG19 успішно з
```

Процес навчання: Навчання SRGAN є змагальним. Я по чергово тренував дискримінатор (показуючи йому справжні та фейкові зображення) та генератор. Ключовою особливістю SRGAN є функція втрат генератора. Вона складається з двох частин:

1. Adversarial Loss: Втрата, яка показує, наскільки добре генератор "обдурих" дискримінатор.
2. Content Loss (Perceptual Loss): Це найголовніше. Замість простого порівняння пікселів (MSE), я пропускав справжнє та згенероване зображення через попередньо навчену мережу VGG19 (без верхніх шарів) і порівнював їхні внутрішні "карти ознак". Це змушує генератор фокусуватися на змісті та текстурі (щоб зображення виглядало як знак), а не просто намагатися вгадати середні значення пікселів (що призводить до розмитості).

Я використовував оптимізатор Adam з `learning_rate=0.0002` та `beta_1=0.5` для обох мереж, як і було рекомендовано. Навчання тривало 20 епох з `batch_size=128`, але потім я довчив модель до 50 епох.

```
--- Початок навчання SRGAN ---

--- Callback для зображень створено. Збереження в './gan_images_v2' ---
Epoch 1/20
46/46 ----- 0s 2s/step - d_loss: 0.2348 - g_loss: 1.4193
Епоха 0: Генерація тестових зображень...
Тестові зображення збережено в: ./gan_images_v2/epoch_0.png
46/46 ----- 226s 3s/step - d_loss: 0.2316 - g_loss: 1.4109
Epoch 2/20
46/46 ----- 49s 983ms/step - d_loss: 0.9102 - g_loss: 0.6626
Epoch 20/20
46/46 ----- 0s 973ms/step - d_loss: 0.0128 - g_loss: 0.1886
--- Навчання завершено ---
--- Початок донавчання ---

--- Callback для зображень створено. Збереження в './gan_images_v2' ---
Epoch 21/50
46/46 ----- 0s 993ms/step - d_loss: 0.0626 - g_loss: 0.2101
Epoch 50/50
46/46 ----- 48s 970ms/step - d_loss: 0.0317 - g_loss: 0.1645
--- Доновчання завершено ---
```

```

# --- 4. Навчання моделі (SRGAN) ---
class SRGAN(Model):
    def __init__(self, generator, discriminator, vgg):
        super(SRGAN, self).__init__()
        self.generator = generator
        self.discriminator = discriminator
        self.vgg = vgg
        self.d_optimizer = Adam(learning_rate=0.0002, beta_1=0.5)
        self.g_optimizer = Adam(learning_rate=0.0002, beta_1=0.5)
        self.g_loss_tracker = tf.keras.metrics.Mean(name="g_loss")
        self.d_loss_tracker = tf.keras.metrics.Mean(name="d_loss")
        print("--- Модель SRGAN ініціалізована ---")

    @property
    def metrics(self):
        return [self.g_loss_tracker, self.d_loss_tracker]

    def compile(self, **kwargs):
        super(SRGAN, self).compile(**kwargs)
        print("--- Модель SRGAN скомпільована ---")

    def train_step(self, data):
        lr_imgs, hr_imgs = data
        batch_size = tf.shape(lr_imgs)[0]
        real_labels = tf.ones((batch_size, 1))
        fake_labels = tf.zeros((batch_size, 1))
        # --- Навчання Дискримінатора ---
        sr_imgs_for_d = self.generator(lr_imgs, training=True)

        with tf.GradientTape() as d_tape:
            real_output = self.discriminator(hr_imgs, training=True)
            fake_output = self.discriminator(sr_imgs_for_d, training=True)

            d_loss_real = bce_loss(real_labels, real_output)
            d_loss_fake = bce_loss(fake_labels, fake_output)
            d_loss = (d_loss_real + d_loss_fake) / 2.0

        grads_d = d_tape.gradient(d_loss,
self.discriminator.trainable_variables)
        self.d_optimizer.apply_gradients(zip(grads_d,
self.discriminator.trainable_variables))
        # --- Навчання Генератора ---
        misleading_labels = tf.ones((batch_size, 1))
        with tf.GradientTape() as g_tape:
            # 1. Повторно генеруємо зображення (тепер стрічка це бачить)
            sr_imgs_for_g = self.generator(lr_imgs, training=True)
            # 2. Perceptual Loss (VGG)
            sr_features = self.vgg(sr_imgs_for_g, training=False)
            hr_features = self.vgg(hr_imgs, training=False)
            content_loss = mse_loss(hr_features, sr_features)
            # 3. Adversarial Loss (пропускаємо через дискримінатор)
            fake_output_g = self.discriminator(sr_imgs_for_g, training=True)
            adversarial_loss = bce_loss(misleading_labels, fake_output_g)
            # 4. Загальна втрата Генератора
            g_loss = content_loss * 0.006 + adversarial_loss * 1e-3

```

```

# Оновлюємо ваги ТІЛЬКИ Генератора
grads_g = g_tape.gradient(g_loss,
self.generator.trainable_variables)
self.g_optimizer.apply_gradients(zip(grads_g,
self.generator.trainable_variables))
# Оновлюємо метрики
self.g_loss_tracker.update_state(g_loss)
self.d_loss_tracker.update_state(d_loss)

return {"g_loss": self.g_loss_tracker.result(), "d_loss":
self.d_loss_tracker.result()}

```

4. Оцінка якості

Оскільки для SRGAN немає простих метрик типу "ассурасу", оцінка якості проводилася переважно візуально. Я зберігав зразки згенерованих зображень кожні 5 епох, після початкового навчання в 20 епох, зображення для порівняння донавчання бралися інші для різноманітності.

Давайте розглянемо перші 20 епох.

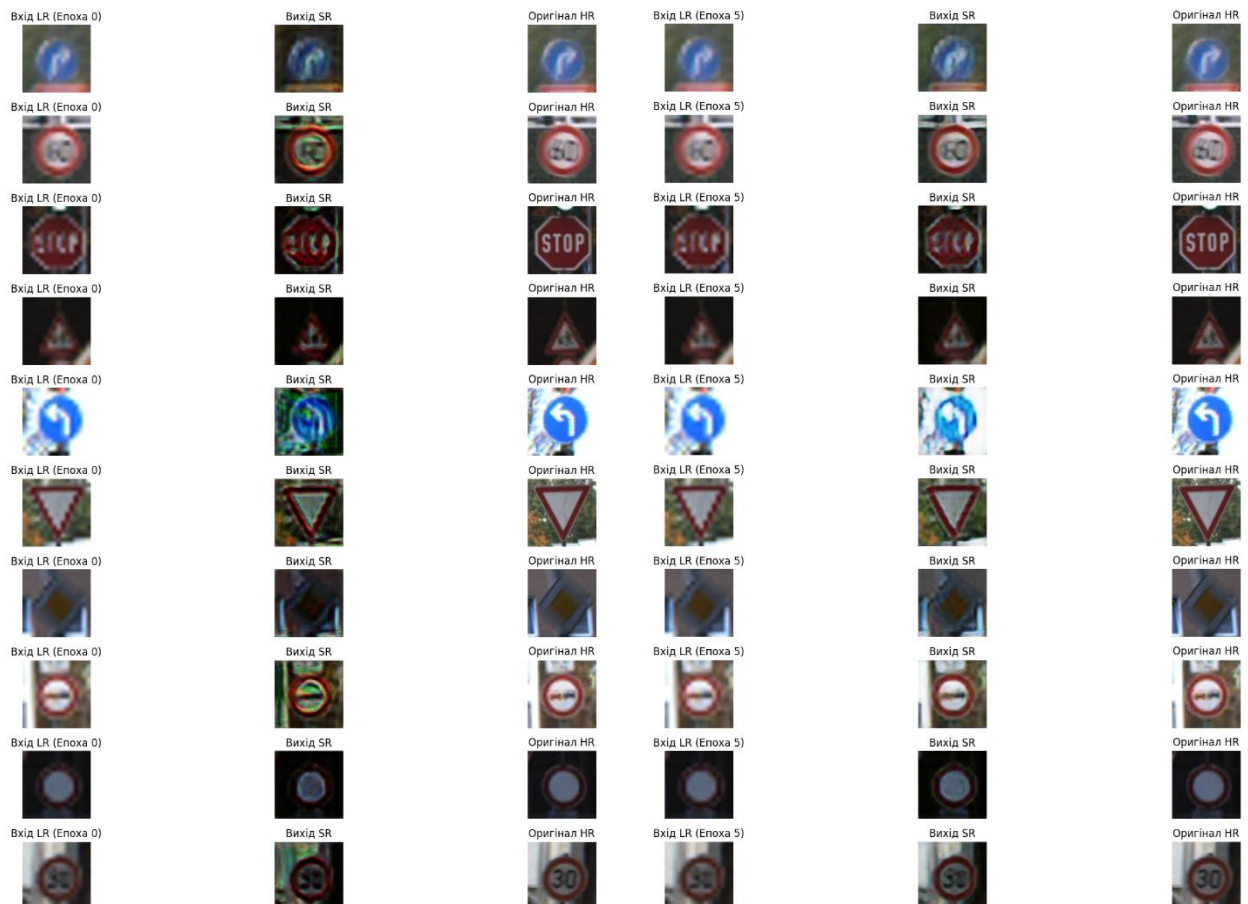


Рис. 1 і 2. Порівняння генерації моделі на 0 і 5 епох

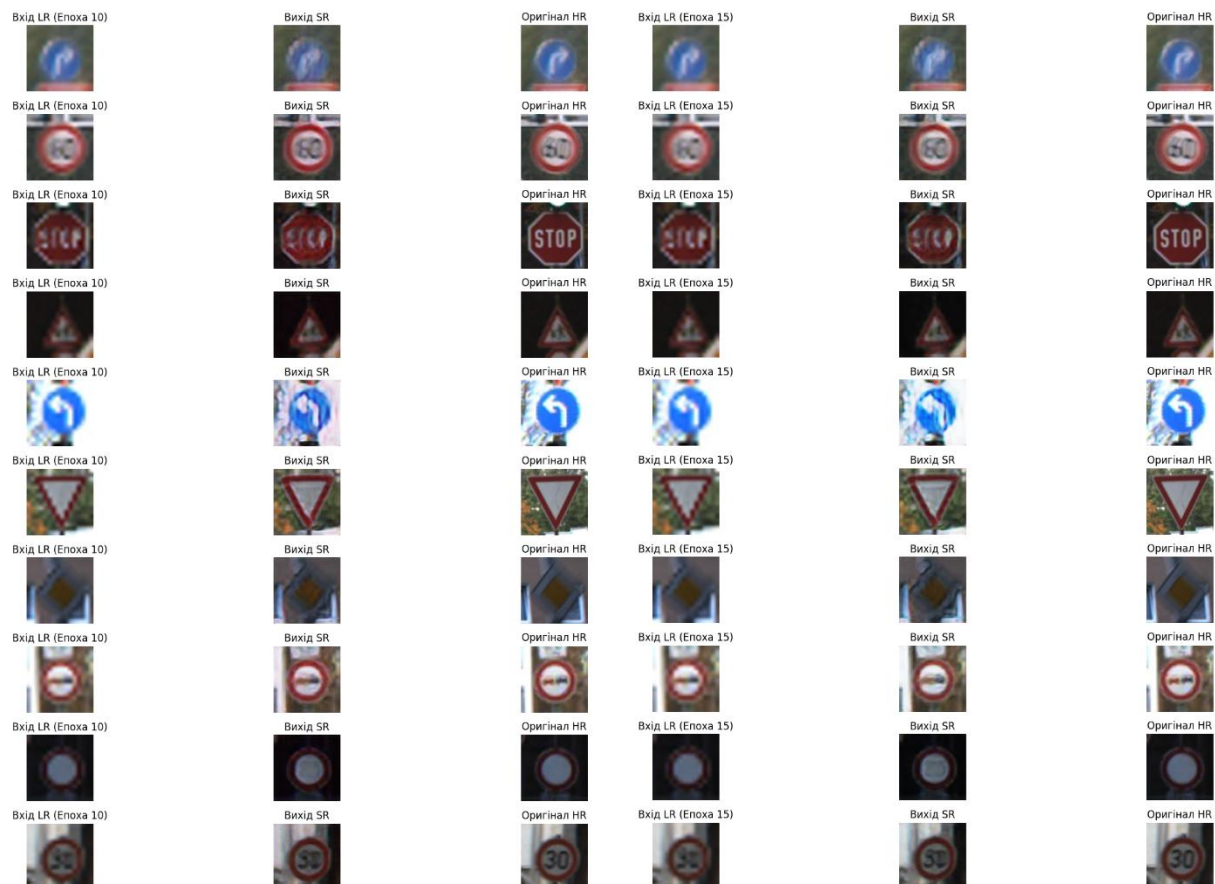


Рис. 3 і 4. Порівняння генерації моделі на 10 і 15 епосі

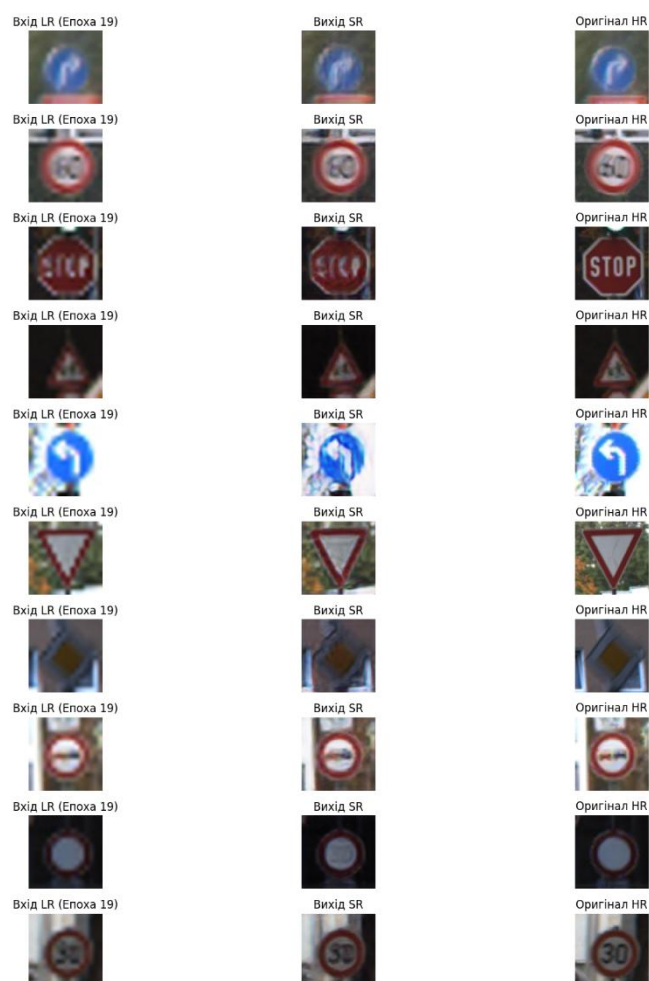


Рис. 5. Порівняння генерації моделі на 20 епосі

В результаті можемо бачити чіткий прогрес. На 0-й епосі (Рис. 1) згенероване зображення (SR) практично не відрізняється від вхідного зображення низької якості (LR), яке було просто збільшене методом бікубічної інтерполяції. Воно таке ж розмите. Вже на 5-й епосі (Рис. 2) модель починає "вгадувати" ключові ознаки – червоний обідок стає чіткішим, а фон стабілізується.

На 10-й та 15-й епохах (Рис. 3, 4) відбувається найбільший стрибок у якості. Це етап, де `content_loss` (втрата змісту) змушує генератор відновлювати символи. Цифри на знаках, які були нерозбірливою прямою, стають більш-менш чіткими та впізнаваними. До 20-ї епохи (Рис. 5) ми маємо вже непоганий результат, який трішки перевершує вхідне LR-зображення.

Тепер, так як ми донавчаємо модель і нові зображення до 50 епохи

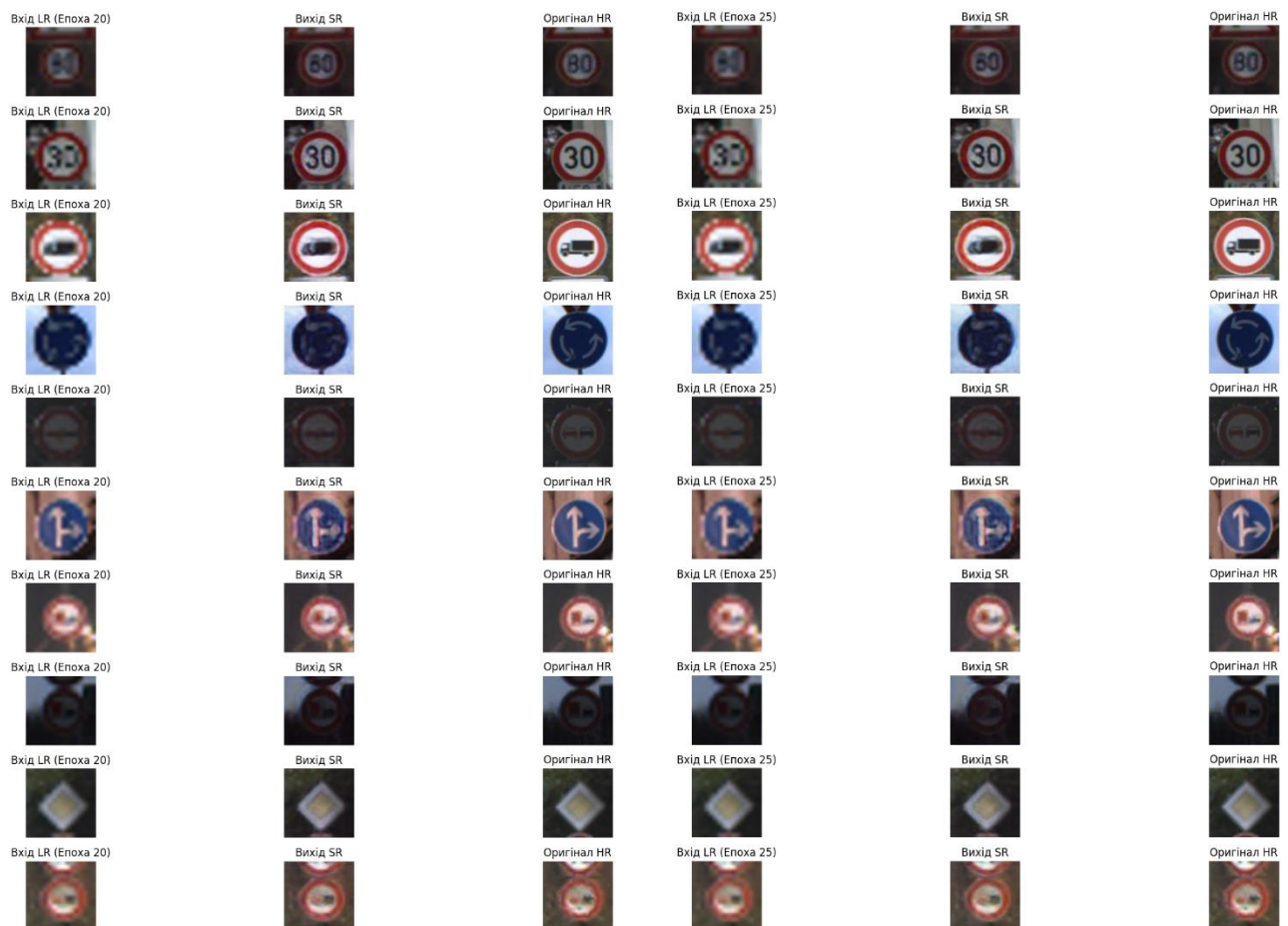


Рис. 6 і 7. Порівняння генерації моделі на 21 і 25 епосі

Після розгляду зображень можна зробити висновок, що відбуваються кардинальні зміни у змісті, як от чіткості цифр. Етап "донавчання" з 20-ї по 50-ту епоху – це етап "полірування". Тут вже працює переважно `adversarial_loss` (змагальна втрата). Генератор вчиться створювати більш фотореалістичні текстури, правильні відблиски та усувати дрібні артефакти, щоб краще "обманювати" дискримінатор.

Також, для достовірної перевірки я порівняв генератора на інших зображеннях, результати видно на Рис. 12. Можемо бачити, що навіть з іншими типами зображень модель змогла досить якісно покращити зображення. Звичайно, що через неймовірно малий розмір неякісного зображення 16x16 важко доповнити його, але навіть з таким обмеження результати непогані, хоча деякі знаки розглянути далі важко, або навіть неможливо. Для майбутнього покращення моделі потрібні хороші навчальні дані з більшою роздільною здатністю.

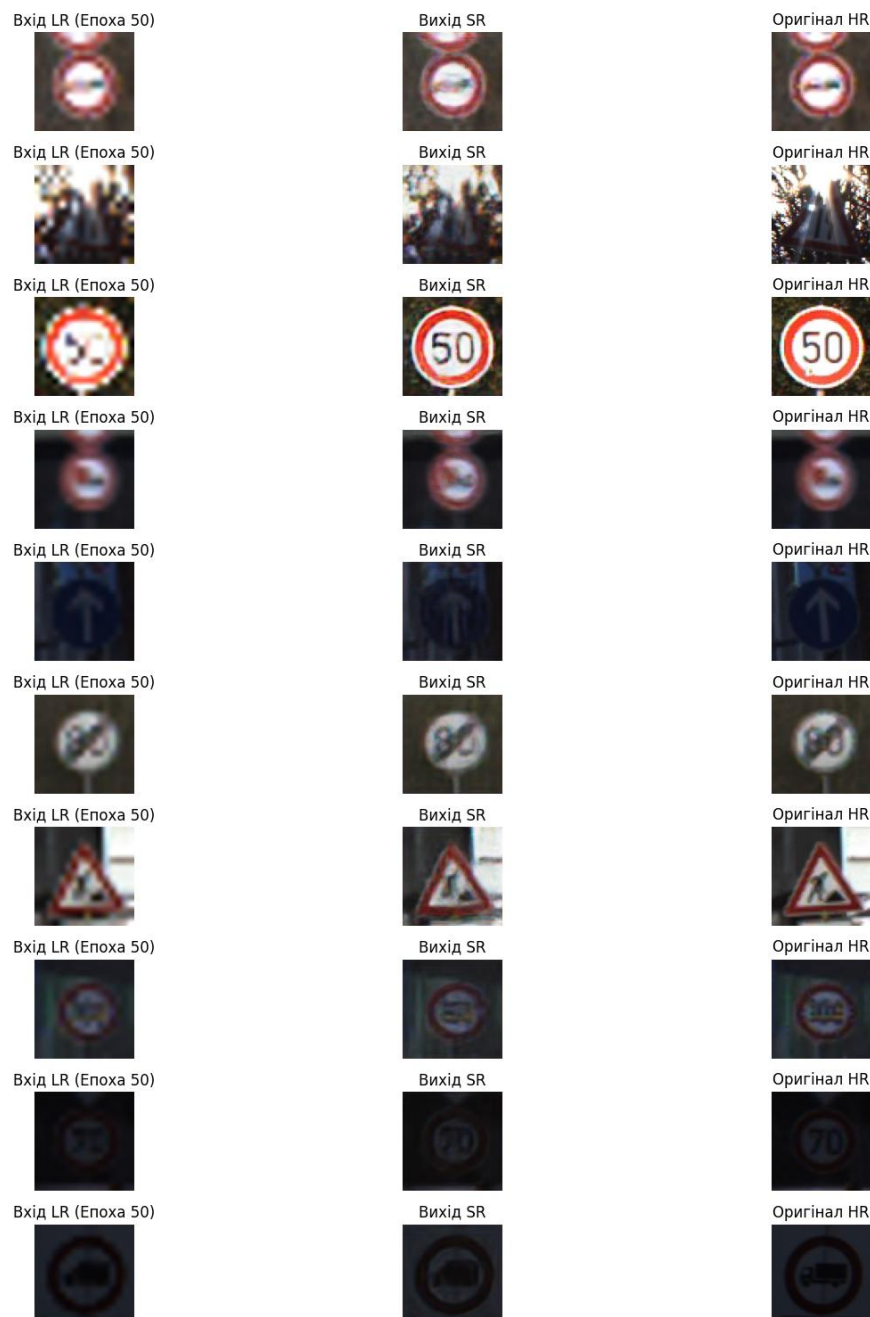
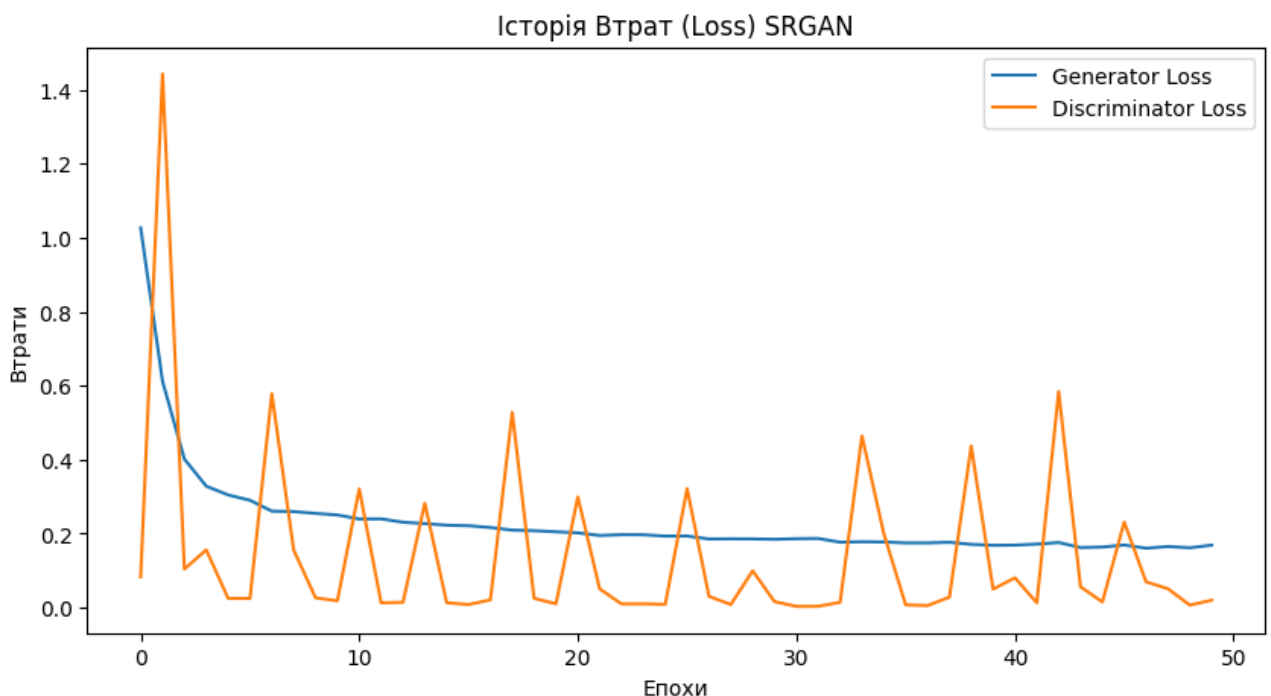


Рис. 12. Порівняння генерації моделі на 50 епосі

5. Побудова візуалізацій

Я побудував два типи візуалізацій:

1. Графіки втрат: Зміни D_loss (втрати дискримінатора) та G_loss (втрати генератора) протягом епох. В ідеалі вони мають досягти певного балансу. В нас повільно спадаючий генератор і стрибучий дискримінатор, що в принципі подібне до очікуваного мною результату.
2. Порівняльні зображення: Я створив сітку зображень, де для кожного знаку показано три версії: [Зображення розмитого дорожнього знака] "Вхід (LR)", "Згенеровано (SR)" та "Оригінал (HR)". Її було розглянуто в минулому пункті.



```
# --- 5. Візуалізація ---
class ImageCallback(tf.keras.callbacks.Callback):
    def __init__(self, test_dataset, save_dir="./gan_images_v2"):
        super(ImageCallback, self).__init__()
        self.test_data = next(iter(test_dataset)) # Беремо один батч з
ТЕСТОВОГО набору
        self.save_dir = save_dir
        if not os.path.exists(self.save_dir):
            os.makedirs(self.save_dir)
        print(f"\n--- Callback для зображень створено. Збереження в
'{self.save_dir}' ---")

    def on_epoch_end(self, epoch, logs=None):
        if epoch % 5 == 0 or epoch == (EPOCHS - 1): # Кожні 5 епох + остання
            print(f"\nЕпоха {epoch}: Генерація тестових зображень...")
            lr_imgs, hr_imgs = self.test_data
            sr_imgs = self.model.generator(lr_imgs, training=False)

            # Де-нормалізація з [-1, 1] до [0, 1]
            lr_imgs = (lr_imgs + 1.0) / 2.0
```



```

sr_imgs = (sr_imgs + 1.0) / 2.0
hr_imgs = (hr_imgs + 1.0) / 2.0

plt.figure(figsize=(15, 15))
num_images_to_show = 10
for i in range(num_images_to_show):
    plt.subplot(num_images_to_show, 3, i*3 + 1)
    plt.imshow(lr_imgs[i*10])
    plt.title(f"Вхід LR (Епоха {epoch})")
    plt.axis('off')

    plt.subplot(num_images_to_show, 3, i*3 + 2)
    plt.imshow(sr_imgs[i*10])
    plt.title("Вихід SR")
    plt.axis('off')

    plt.subplot(num_images_to_show, 3, i*3 + 3)
    plt.imshow(hr_imgs[i*10])
    plt.title("Оригінал HR")
    plt.axis('off')

plt.tight_layout()
save_path = os.path.join(self.save_dir, f"epoch_{epoch}.png")
plt.savefig(save_path)
print(f"Тестові зображення збережено в: {save_path}")
plt.close()

```

6. Аналіз виконання програми

Візуальний аналіз показав, що модель SRGAN успішно впоралася із завданням. Згенеровані SR-зображення були значно чіткішими та деталізованішими, ніж розмиті LR-входи. Модель навчилася відновлювати контури знаків, цифри та символи на них.

Основною проблемою під час навчання GAN є нестабільність. Іноді дискримінатор стає "занадто розумним" і його втрати стрімко падають до нуля, через що генератор перестає навчатися (оскільки дискримінатор миттєво розпізнає будь-яку підробку). "Колапс мод", коли генератор починає створювати лише один тип зображень, у моєму випадку не спостерігався, оскільки вхід генератора (LR-зображення) був різноманітним, на відміну від DCGAN, де вхід – випадковий шум.

Потенціал величезний. Якщо у нас є датасет з багатьма низькоякісними зображеннями (напр., знятими в русі або здалеку), ми можемо використати SRGAN, щоб "покращити" їх до подачі в класифікатор. Це може значно підвищити точність класифікації, оскільки класифікатор отримає більш чіткі ознаки.

Можна спробувати глибшу архітектуру генератора (більше залишкових блоків) або використати складнішу perceptual loss, беручи ознаки з кількох шарів VGG19.

Висновки

У ході виконання лабораторної роботи було вивчено теоретичні основи генеративно-змагальних мереж та реалізовано просунуту архітектуру SRGAN. Я успішно навчив модель покращувати роздільну здатність зображень дорожніх знаків з датасету GTSRB. Аналіз візуальних результатів та втрат показав ефективність моделі. Я дослідив ключові компоненти SRGAN, зокрема складну функцію втрат (Perceptual Loss), яка є вирішальною для генерації фотореалістичних зображень. Робота підтвердила великий потенціал SRGAN для задач аугментації даних шляхом відновлення якості зображень.